Métodos de Machine Learning para Clasificación

Edison Achalma
Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Este abstract será actualizado una vez que se complete el contenido final del artículo.

Tabla de contenidos

Introduction **4**

1 Introducción al aprendizaje supervisado **4**
1.1 Concepto fundamental . . . 4
1.2 Tipos de problemas de clasificación . . . 4
1.3 Dataset de ejemplo: Iris . . . 5

2 Regresión logística **6**
2.1 Fundamento matemático . . . 6
2.1.1 Modelo de probabilidad . . . 6
2.1.2 Función sigmoide . . . 6
2.1.3 Odds y log-odds (logit) . . . 7
2.1.4 Interpretación de coeficientes . . . 7
2.1.5 Estimación: Maximum Likelihood . . . 8
2.1.6 Frontera de decisión . . . 8
2.1.7 Ventajas . . . 8
2.1.8 Limitaciones . . . 8
2.1.9 Aplicaciones económicas . . . 8
2.2 Implementación . . . 8
2.3 Predicción e interpretación . . . 9
2.4 Visualización del modelo . . . 10

Edison Achalma https://orcid.org/0000-0001-6996-3364

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; https://credit.niso.org/) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

3 K-Nearest Neighbors (KNN) 11
3.1 Fundamento del algoritmo . . . 11
3.1.1 Principio fundamental . . . 11
3.1.2 Definición formal . . . 11
3.1.3 Métricas de distancia . . . 12
3.1.4 Regla de decisión probabilística . . . 12
3.1.5 Versión ponderada . . . 12
3.1.6 Parámetro k: Trade-off sesgo-varianza . . . 13
3.1.7 Complejidad computacional . . . 13
3.1.8 Maldición de la dimensionalidad . . . 13
3.1.9 Sensibilidad a la escala . . . 14
3.1.10 Fronteras de decisión . . . 14
3.1.11 Propiedades teóricas . . . 14
3.1.12 Ventajas . . . 14
3.1.13 Limitaciones . . . 14
3.1.14 Selección del parámetro k . . . 15
3.1.15 Extensiones y variantes . . . 15
3.1.16 Aplicaciones en economía . . . 15
3.1.17 Comparación con métodos paramétricos . . . 15
3.2 Implementación . . . 15
3.3 Visualización del efecto de k . . . 16
3.4 Visualización de fronteras de decisión . . . 17

4 Árboles de decisión 18
4.1 Fundamento del algoritmo . . . 18
4.1.1 Estructura del árbol . . . 18
4.1.2 Algoritmo de construcción recursiva . . . 18
4.1.3 Criterios de impureza . . . 19
4.1.4 Comparación Gini vs Entropy . . . 20
4.1.5 Criterios de parada . . . 20
4.1.6 Predicción . . . 21
4.1.7 Fronteras de decisión . . . 21
4.1.8 Importancia de características . . . 21
4.1.9 Poda del árbol (Pruning) . . . 21
4.1.10 Ventajas . . . 22
4.1.11 Limitaciones . . . 22
4.1.12 Hiperparámetros principales . . . 22
4.1.13 Aplicaciones en economía . . . 23
4.1.14 Comparación con otros métodos . . . 23
4.1.15 Extensiones y variantes . . . 23
4.2 Implementación básica . . . 23
4.3 Visualización del árbol . . . 24
4.4 Efecto de la profundidad . . . 24

5 Random Forests 26
5.1 Fundamento del algoritmo . . . 26
5.1.1 Concepto central: Ensemble Learning . . . 26
5.1.2 Algoritmo Random Forest . . . 26

5.1.3 Dos fuentes de aleatoriedad . . . 27
5.1.4 Relación con Bagging . . . 27
5.1.5 Out-of-Bag (OOB) Error . . . 27
5.1.6 Importancia de características . . . 27
5.1.7 Propiedades teóricas . . . 28
5.1.8 Hiperparámetros principales . . . 28
5.1.9 Ventajas . . . 28
5.1.10 Limitaciones . . . 29
5.1.11 Comparación con otros métodos . . . 29
5.1.12 Selección de hiperparámetros . . . 29
5.1.13 Aplicaciones en economía . . . 30
5.1.14 Variantes y extensiones . . . 30
5.2 Implementación . . . 30
5.3 Efecto del número de árboles . . . 31

6 Clasificación multi-clase 32
6.1 Dataset con 3 clases . . . 32
6.2 Entrenar modelos multi-clase . . . 33
6.3 Visualización multi-clase . . . 34

7 Comparación de modelos 35
7.1 Curvas ROC para cada modelo . . . 35
7.2 Tabla comparativa de métricas . . . 36

8 Aplicaciones económicas 37
8.1 Aplicación 1: Clasificación de riesgo crediticio . . . 37
8.2 Aplicación 2: Segmentación de clientes . . . 39

9 Ejercicios prácticos 41
9.1 Ejercicio: Optimización de hiperparámetros . . . 41

10 Conclusión 43

11 Próximos pasos 43

12 Recursos adicionales 43

13 Publicaciones Similares 44

Métodos de Machine Learning para Clasificación

En esta novena guía exploraremos los principales algoritmos de machine learning para problemas de clasificación. Estos métodos son fundamentales para resolver problemas económicos como clasificación de riesgo crediticio, predicción de comportamiento de consumidores, segmentación de mercados, entre otros.

1 Introducción al aprendizaje supervisado

1.1 Concepto fundamental

El aprendizaje supervisado es un tipo de machine learning donde el modelo aprende de datos etiquetados (con respuestas conocidas) para predecir etiquetas de nuevos datos.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.colors import ListedColormap

# Scikit-learn
from sklearn.datasets import load_iris, make_classification
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (confusion_matrix, classification_report,
                             roc_curve, auc, accuracy_score)

# Modelos
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# Visualización de árboles
from sklearn import tree

# Configuración
plt.style.use('seaborn-v0_8-darkgrid')
plt.rcParams['figure.figsize'] = (12, 6)
np.random.seed(42)

print("Librerías importadas exitosamente")
```

1.2 Tipos de problemas de clasificación

```
print("""
TIPOS DE PROBLEMAS DE CLASIFICACIÓN

1. CLASIFICACIÓN BINARIA
```

```
   - Dos clases posibles
   - Ejemplos económicos:
     * Aprobado/Rechazado (crédito)
     * Paga/No paga (incumplimiento)
     * Éxito/Fracaso (empresa)
     * Compra/No compra (marketing)

2. CLASIFICACIÓN MULTI-CLASE
   - Más de dos clases
   - Ejemplos económicos:
     * Clasificación de riesgo (AAA, AA, A, BBB, etc.)
     * Segmentación de clientes (bajo, medio, alto valor)
     * Categorías de productos
     * Niveles socioeconómicos (A, B, C, D, E)

3. CLASIFICACIÓN MULTI-ETIQUETA
   - Múltiples etiquetas simultáneas
   - Ejemplos económicos:
     * Características de consumidor (múltiples intereses)
     * Atributos de producto (múltiples categorías)
""")
```

1.3 Dataset de ejemplo: Iris

```
# Cargar dataset clásico
iris = load_iris()

# Crear DataFrame
df_iris = pd.DataFrame(
    data=iris.data,
    columns=iris.feature_names
)
df_iris['especie'] = iris.target
df_iris['especie_nombre'] = df_iris['especie'].map({
    0: 'Setosa',
    1: 'Versicolor',
    2: 'Virginica'
})

print("Dataset Iris:")
print(df_iris.head())
print(f"\nForma: {df_iris.shape}")
print(f"\nDistribución de especies:")
print(df_iris['especie_nombre'].value_counts())

# Visualizar (usar solo 2 características)
plt.figure(figsize=(10, 6))
```

```
for especie in df_iris['especie'].unique():
    mask = df_iris['especie'] == especie
    plt.scatter(
        df_iris.loc[mask, 'sepal length (cm)'],
        df_iris.loc[mask, 'sepal width (cm)'],
        label=df_iris.loc[mask, 'especie_nombre'].iloc[0],
        alpha=0.7,
        s=100
    )

plt.xlabel('Largo del sépalo (cm)')
plt.ylabel('Ancho del sépalo (cm)')
plt.title('Dataset Iris - Clasificación de especies')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('iris_scatter.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'iris_scatter.png'")
```

2 Regresión logística

2.1 Fundamento matemático

La regresión logística es uno de los métodos más fundamentales y ampliamente utilizados para problemas de clasificación binaria. A pesar de su nombre que incluye "regresión", es un algoritmo de clasificación que predice probabilidades de pertenencia a clases mediante una transformación no lineal de una combinación lineal de características.

2.1.1 Modelo de probabilidad

Para clasificación binaria donde $Y \in \{0, 1\}$, la regresión logística modela la probabilidad de que la observación pertenezca a la clase positiva ($Y = 1$):

$$P(Y = 1|\mathbf{X}) = p(\mathbf{X}) = \frac{1}{1 + e^{-(\beta_0+\beta_1 X_1+\cdots+\beta_p X_p)}}$$

Definiendo la combinación lineal:

$$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p = \beta_0 + \mathbf{X}^\top \boldsymbol{\beta}$$

La probabilidad se expresa como:

$$p(\mathbf{X}) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z}$$

Esta función se conoce como **función logística** o **sigmoide**.

2.1.2 Función sigmoide

La función sigmoide tiene la forma:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Propiedades clave:

1. **Rango**: $\sigma(z) \in (0, 1)$ para todo $z \in \mathbb{R}$

   - Garantiza que las predicciones sean probabilidades válidas

2. **Simetría**: $\sigma(-z) = 1 - \sigma(z)$

3. **Límites asintóticos**:
$$\lim_{z\to\infty} \sigma(z) = 1, \quad \lim_{z\to-\infty} \sigma(z) = 0$$

4. **Punto medio**: $\sigma(0) = 0.5$

5. **Derivada**:
$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$
Esta propiedad simplifica el cálculo de gradientes.

2.1.3 Odds y log-odds (logit)

Odds (momios):
La probabilidad de éxito sobre la probabilidad de fracaso:

$$\text{Odds} = \frac{p(\mathbf{X})}{1 - p(\mathbf{X})} = e^z$$

Log-odds (logit):
Tomando logaritmo natural:

$$\log\left(\frac{p(\mathbf{X})}{1 - p(\mathbf{X})}\right) = z = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$$

Esta es la **transformación logit** que convierte probabilidades (0,1) al rango completo de números reales $(-\infty, \infty)$.

Interpretación crucial: El modelo es lineal en el log-odds, no en la probabilidad misma.

2.1.4 Interpretación de coeficientes

Un incremento unitario en X_j (manteniendo otras variables constantes):
En log-odds:
$$\Delta \log(\text{Odds}) = \beta_j$$

En odds:
$$\text{Odds ratio} = e^{\beta_j}$$

- Si $\beta_j > 0$: $e^{\beta_j} > 1 \rightarrow$ La variable incrementa el odds de $Y = 1$
- Si $\beta_j < 0$: $e^{\beta_j} < 1 \rightarrow$ La variable reduce el odds de $Y = 1$
- Si $\beta_j = 0$: $e^{\beta_j} = 1 \rightarrow$ La variable no afecta el odds

Efecto marginal en probabilidad:

$$\frac{\partial p}{\partial X_j} = \beta_j \cdot p(\mathbf{X}) \cdot (1 - p(\mathbf{X}))$$

El efecto no es constante; depende del nivel de probabilidad actual.

2.1.5 Estimación: Maximum Likelihood

Los coeficientes se estiman mediante **máxima verosimilitud** (MLE).
Log-verosimilitud:

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^{n} [Y_i \log p(\mathbf{X}_i) + (1 - Y_i) \log(1 - p(\mathbf{X}_i))]$$

Objetivo: Maximizar $\ell(\boldsymbol{\beta})$ mediante métodos iterativos (Newton-Raphson, gradient descent).

Función de pérdida equivalente (Binary Cross-Entropy):

$$\text{BCE}(\boldsymbol{\beta}) = -\frac{1}{n} \sum_{i=1}^{n} [Y_i \log \hat{p}_i + (1 - Y_i) \log(1 - \hat{p}_i)]$$

2.1.6 Frontera de decisión

La frontera de decisión es donde $p(\mathbf{X}) = 0.5$:

$$\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p = 0$$

En el espacio de características, esta es una **hipersuperficie lineal** (hiperplano).

2.1.7 Ventajas

- Salidas probabilísticas interpretables
- Eficiencia computacional
- Pocos hiperparámetros
- Funciona bien como baseline
- Extensible a clasificación multi-clase

2.1.8 Limitaciones

- Asume linealidad en log-odds
- No captura relaciones no lineales complejas
- Sensible a outliers
- Requiere features relevantes

2.1.9 Aplicaciones económicas

- Predicción de incumplimiento crediticio
- Participación laboral
- Decisiones de compra
- Migración
- Adopción de tecnología
- Éxito de startups

2.2 Implementación

```
# Preparar datos (clasificación binaria: Virginica vs resto)
X = iris.data[:, :2]  # Solo usar 2 características para visualización
y = (iris.target == 2).astype(int)  # 1 si es Virginica, 0 si no

# Dividir datos
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

print(f"Conjunto de entrenamiento: {X_train.shape[0]} muestras")
print(f"Conjunto de prueba: {X_test.shape[0]} muestras")
print(f"\nDistribución en entrenamiento:")
print(f"  Clase 0 (No Virginica): {(y_train == 0).sum()}")
print(f"  Clase 1 (Virginica): {(y_train == 1).sum()}")

# Entrenar modelo
modelo_logit = LogisticRegression(random_state=42)
modelo_logit.fit(X_train, y_train)

print(f"\n\nMODELO ENTRENADO")
print("=" * 70)
print(f"Intercepto ( ): {modelo_logit.intercept_[0]:.4f}")
print(f"Coeficientes:")
print(f"    (largo sépalo): {modelo_logit.coef_[0, 0]:.4f}")
print(f"    (ancho sépalo): {modelo_logit.coef_[0, 1]:.4f}")
```

2.3 Predicción e interpretación

```
# Predecir probabilidades
probs_train = modelo_logit.predict_proba(X_train)[:, 1]
probs_test = modelo_logit.predict_proba(X_test)[:, 1]

# Predecir clases
y_pred = modelo_logit.predict(X_test)

# Evaluar
accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy en test: {accuracy:.4f} ({accuracy*100:.1f}%)")

# Matriz de confusión
cm = confusion_matrix(y_test, y_pred)
print(f"\nMatriz de confusión:")
print(cm)

# Reporte de clasificación
print(f"\nReporte de clasificación:")
print(classification_report(y_test, y_pred,
```

```
                          target_names=['No Virginica', 'Virginica']))

# Ejemplo de predicción
ejemplo = np.array([[6.5, 3.0]])  # Nuevo ejemplo
prob = modelo_logit.predict_proba(ejemplo)[0, 1]
pred = modelo_logit.predict(ejemplo)[0]

print(f"\n\nEJEMPLO DE PREDICCIÓN")
print(f"Características: Largo={ejemplo[0, 0]} cm, Ancho={ejemplo[0, 1]} cm")
print(f"Probabilidad de ser Virginica: {prob:.4f} ({prob*100:.1f}%)")
print(f"Predicción: {'Virginica' if pred == 1 else 'No Virginica'}")
```

2.4 Visualización del modelo

```
def plot_decision_boundary(modelo, X, y, title, filename):
    """Visualiza la frontera de decisión del modelo"""

    h = 0.02  # Tamaño del paso en la malla

    # Crear malla
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                         np.arange(y_min, y_max, h))

    # Predecir para cada punto en la malla
    Z = modelo.predict_proba(np.c_[xx.ravel(), yy.ravel()])[:, 1]
    Z = Z.reshape(xx.shape)

    # Graficar
    plt.figure(figsize=(12, 8))

    # Contorno de probabilidades
    contour = plt.contourf(xx, yy, Z, levels=20, cmap='RdYlBu_r', alpha=0.6)
    plt.colorbar(contour, label='Probabilidad clase positiva')

    # Frontera de decisión (probabilidad = 0.5)
    plt.contour(xx, yy, Z, levels=[0.5], colors='black', linewidths=2)

    # Puntos de datos
    scatter = plt.scatter(X[:, 0], X[:, 1], c=y,
                          cmap='RdYlBu_r', edgecolors='black',
                          s=100, alpha=0.9)

    plt.xlabel('Largo del sépalo (cm)', fontsize=12)
    plt.ylabel('Ancho del sépalo (cm)', fontsize=12)
    plt.title(title, fontsize=14, fontweight='bold')
```

```
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(filename, dpi=300, bbox_inches='tight')
    print(f"Gráfico guardado como '{filename}'")

# Visualizar
plot_decision_boundary(
    modelo_logit, X_train, y_train,
    'Regresión Logística - Frontera de Decisión',
    'logistic_regression_boundary.png'
)
```

3 K-Nearest Neighbors (KNN)

3.1 Fundamento del algoritmo

K-Nearest Neighbors es un algoritmo de clasificación no paramétrico basado en la premisa de que observaciones similares (vecinas) tienden a pertenecer a la misma clase. Es uno de los algoritmos más simples conceptualmente, pero sorprendentemente efectivo en muchas aplicaciones.

3.1.1 Principio fundamental

Idea central: "Dime quiénes son tus vecinos y te diré quién eres"

Una nueva observación se clasifica según la clase más común entre sus k vecinos más cercanos en el espacio de características.

3.1.2 Definición formal

Dado:

- Conjunto de entrenamiento $\mathcal{D} = \{(\mathbf{X}_i, Y_i)\}_{i=1}^{n}$
- Nuevo punto a clasificar $\mathbf{X}_0$
- Número de vecinos k

El algoritmo:

1. **Calcular distancias** a todos los puntos de entrenamiento:

$$d_i = d(\mathbf{X}_0, \mathbf{X}_i) \quad \text{para } i = 1, \ldots, n$$

2. **Identificar** los k puntos con menores distancias:

$$\mathcal{N}_k(\mathbf{X}_0) = \{k \text{ vecinos más cercanos}\}$$

3. **Votar por mayoría**:

$$\hat{Y}_0 = \arg\max_c \sum_{i \in \mathcal{N}_k(\mathbf{X}_0)} \mathbb{1}(Y_i = c)$$

donde $\mathbb{1}(\cdot)$ es la función indicadora y c recorre todas las clases posibles.

3.1.3 Métricas de distancia

La elección de la métrica de distancia es fundamental en KNN:
1. Distancia Euclidiana (la más común):

$$d(\mathbf{X}_i, \mathbf{X}_j) = \sqrt{\sum_{m=1}^{p} (X_{im} - X_{jm})^2} = \|\mathbf{X}_i - \mathbf{X}_j\|_2$$

2. Distancia Manhattan (city-block):

$$d(\mathbf{X}_i, \mathbf{X}_j) = \sum_{m=1}^{p} |X_{im} - X_{jm}| = \|\mathbf{X}_i - \mathbf{X}_j\|_1$$

3. Distancia Minkowski (generalización):

$$d(\mathbf{X}_i, \mathbf{X}_j) = \left(\sum_{m=1}^{p} |X_{im} - X_{jm}|^q \right)^{1/q}$$

donde:

- $q = 1$: Manhattan
- $q = 2$: Euclidiana
- $q \to \infty$: Chebyshev

4. Distancia de Mahalanobis (considera correlaciones):

$$d(\mathbf{X}_i, \mathbf{X}_j) = \sqrt{(\mathbf{X}_i - \mathbf{X}_j)^\top \mathbf{\Sigma}^{-1} (\mathbf{X}_i - \mathbf{X}_j)}$$

3.1.4 Regla de decisión probabilística

KNN puede proporcionar probabilidades estimadas:

$$P(Y = c | \mathbf{X}_0) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{X}_0)} \mathbb{1}(Y_i = c)$$

Es decir, la proporción de vecinos que pertenecen a la clase c.

3.1.5 Versión ponderada

En lugar de votos iguales, se pueden usar **pesos inversamente proporcionales a la distancia**:

$$\hat{Y}_0 = \arg\max_c \sum_{i \in \mathcal{N}_k(\mathbf{X}_0)} w_i \cdot \mathbb{1}(Y_i = c)$$

donde:

$$w_i = \frac{1}{d(\mathbf{X}_0, \mathbf{X}_i) + \epsilon}$$

$\epsilon > 0$ evita división por cero. Vecinos más cercanos tienen más influencia.

3.1.6 Parámetro k: Trade-off sesgo-varianza

k pequeño (ej: k=1):

- **Baja bias**: Frontera de decisión flexible, se adapta a detalles locales
- **Alta varianza**: Sensible a ruido, predicciones inestables
- Riesgo de **overfitting**
- Fronteras irregulares

k grande (ej: k=50):

- **Alta bias**: Frontera de decisión suave, ignora detalles locales
- **Baja varianza**: Predicciones estables
- Riesgo de **underfitting**
- Fronteras más regulares

k óptimo: Balance entre sesgo y varianza, determinado típicamente por validación cruzada.

3.1.7 Complejidad computacional

Búsqueda naive:

- **Entrenamiento**: $O(1)$ (solo almacenar datos)
- **Predicción**: $O(np)$ por cada nueva observación
  - Calcular n distancias de dimensión p
  - Encontrar los k mínimos: $O(n \log k)$

Total por predicción: $O(np + n \log k) \approx O(np)$
Estructuras eficientes (KD-trees, Ball trees):

- **Construcción**: $O(np \log n)$
- **Predicción**: $O(\log n)$ en promedio (mejor caso)

Sin embargo, en alta dimensionalidad (p grande), estas estructuras pierden eficiencia.

3.1.8 Maldición de la dimensionalidad

En espacios de alta dimensión, KNN sufre severamente:
Problema 1: Las distancias se vuelven uniformes

$$\lim_{p \to \infty} \frac{d_{\max} - d_{\min}}{d_{\min}} \to 0$$

Todos los puntos parecen equidistantes, haciendo que el concepto de "vecino cercano" pierda significado.

Problema 2: Volumen crece exponencialmente
Para mantener densidad constante de vecinos, el volumen requerido crece como r^p.
Consecuencia: KNN funciona mejor cuando $p < 10 - 20$.

3.1.9 Sensibilidad a la escala

KNN es **extremadamente sensible** a la escala de las variables.

Problema: Una variable con rango [0, 1000] dominará una con rango [0, 1] en el cálculo de distancias.

Solución obligatoria: Estandarización

$$\tilde{X}_{jm} = \frac{X_{jm} - \mu_m}{\sigma_m}$$

Sin estandarización, el algoritmo es inválido.

3.1.10 Fronteras de decisión

Las fronteras de decisión de KNN son:

- **No lineales**: Pueden ser arbitrariamente complejas
- **No paramétricas**: No asumen forma funcional
- **Locales**: Definidas solo por vecinos cercanos
- **Irregulares**: Especialmente con k pequeño

Para clasificación binaria con k=1, la frontera es la **diagrama de Voronoi** del conjunto de entrenamiento.

3.1.11 Propiedades teóricas

Teorema (Cover & Hart, 1967):
Para k=1 y $n \to \infty$, el error de KNN está acotado por:

$$\text{Error}_{1-NN} \leq 2 \cdot \text{Error}_{\text{Bayes}}$$

donde $\text{Error}_{\text{Bayes}}$ es el error del clasificador óptimo.

Consistencia: KNN es consistente si $k \to \infty$ y $k/n \to 0$ cuando $n \to \infty$.

3.1.12 Ventajas

1. **Simplicidad conceptual**: Muy intuitivo
2. **No paramétrico**: No asume distribución de datos
3. **Flexibilidad**: Captura fronteras complejas
4. **Lazy learning**: No hay fase de entrenamiento
5. **Actualización trivial**: Fácil incorporar nuevos datos
6. **Múltiples clases**: Extiende naturalmente a clasificación multi-clase

3.1.13 Limitaciones

1. **Costo computacional alto en predicción**: $O(np)$ por cada predicción
2. **Sensibilidad a escala**: Requiere estandarización obligatoria
3. **Maldición dimensionalidad**: Falla en espacios de alta dimensión
4. **Memoria**: Debe almacenar todo el conjunto de entrenamiento
5. **Sensibilidad a features irrelevantes**: Variables ruidosas contaminan distancias
6. **No interpretable**: No hay modelo explícito para interpretar
7. **Elección de k**: Requiere validación cruzada

3.1.14 Selección del parámetro k

Validación cruzada:

$$k^* = \arg\min_k \frac{1}{K} \sum_{j=1}^{K} \text{Error}_j(k)$$

Regla empírica:

$$k \approx \sqrt{n}$$

Recomendación: Probar valores impares para evitar empates en clasificación binaria.

3.1.15 Extensiones y variantes

1. Weighted KNN: Pesos basados en distancia o kernel.
2. Distance-weighted KNN:

$$w_i = \exp\left(-\frac{d_i^2}{2\sigma^2}\right)$$

3. Adaptive KNN: Diferentes valores de k para diferentes regiones del espacio.
4. Locally Weighted Learning: Combina KNN con regresión local.

3.1.16 Aplicaciones en economía

1. **Sistemas de recomendación**: Recomendar productos basados en consumidores similares
2. **Credit scoring**: Clasificar aplicantes basándose en aplicantes históricos similares
3. **Detección de fraude**: Identificar transacciones anómalas (diferentes de vecinos)
4. **Valuación inmobiliaria**: Precio de casas basado en propiedades similares
5. **Segmentación de clientes**: Agrupar clientes con características similares
6. **Predicción de quiebra**: Clasificar empresas basándose en empresas similares del pasado

3.1.17 Comparación con métodos paramétricos

Aspecto	KNN	Métodos Paramétricos
Supuestos	Mínimos	Forma funcional específica
Interpretabilidad	Baja	Alta
Flexibilidad	Alta	Media-Baja
Costo computacional	Alto (predicción)	Bajo (predicción)
Dimensionalidad	Sufre con p alto	Mejor con regularización
Fronteras	No lineales complejas	Lineales o simples

3.2 Implementación

```
# Estandarizar características (importante para KNN)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Entrenar modelos con diferentes valores de k
k_values = [1, 3, 5, 10, 15, 20]
resultados_knn = []

print("EVALUACIÓN DE KNN CON DIFERENTES VALORES DE K")
print("=" * 70)
print(f"{'K':<5} {'Accuracy Train':<15} {'Accuracy Test':<15} {'Diferencia':<12}")
print("-" * 70)

for k in k_values:
    # Entrenar modelo
    modelo_knn = KNeighborsClassifier(n_neighbors=k)
    modelo_knn.fit(X_train_scaled, y_train)

    # Evaluar
    acc_train = modelo_knn.score(X_train_scaled, y_train)
    acc_test = modelo_knn.score(X_test_scaled, y_test)
    diff = acc_train - acc_test

    resultados_knn.append({
        'k': k,
        'acc_train': acc_train,
        'acc_test': acc_test,
        'diff': diff
    })

    print(f"{k:<5} {acc_train:<15.4f} {acc_test:<15.4f} {diff:<12.4f}")

# Encontrar mejor k
df_knn = pd.DataFrame(resultados_knn)
mejor_k = df_knn.loc[df_knn['acc_test'].idxmax(), 'k']
print(f"\n\nMejor k: {int(mejor_k)} (mayor accuracy en test)")
```

3.3 Visualización del efecto de k

```
# Entrenar modelo con mejor k
modelo_knn_final = KNeighborsClassifier(n_neighbors=int(mejor_k))
modelo_knn_final.fit(X_train_scaled, y_train)

# Graficar accuracy vs k
fig, axes = plt.subplots(1, 2, figsize=(15, 5))
```

```
# Accuracy
axes[0].plot(df_knn['k'], df_knn['acc_train'], 'o-',
             label='Training', linewidth=2, markersize=8)
axes[0].plot(df_knn['k'], df_knn['acc_test'], 's-',
             label='Test', linewidth=2, markersize=8)
axes[0].axvline(x=mejor_k, color='red', linestyle='--',
                label=f'Mejor k={int(mejor_k)}')
axes[0].set_xlabel('Número de vecinos (k)')
axes[0].set_ylabel('Accuracy')
axes[0].set_title('Accuracy vs K')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Overfitting
axes[1].plot(df_knn['k'], df_knn['diff'], 'o-',
             linewidth=2, markersize=8, color='purple')
axes[1].axhline(y=0, color='black', linestyle='--', alpha=0.3)
axes[1].axvline(x=mejor_k, color='red', linestyle='--',
                label=f'Mejor k={int(mejor_k)}')
axes[1].set_xlabel('Número de vecinos (k)')
axes[1].set_ylabel('Train Accuracy - Test Accuracy')
axes[1].set_title('Diferencia Train-Test (Indicador de Overfitting)')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('knn_k_selection.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'knn_k_selection.png'")
```

3.4 Visualización de fronteras de decisión

```
# Comparar diferentes valores de k
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
axes = axes.ravel()

h = 0.02
x_min, x_max = X_train_scaled[:, 0].min() - 1, X_train_scaled[:, 0].max() + 1
y_min, y_max = X_train_scaled[:, 1].min() - 1, X_train_scaled[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))

for idx, k in enumerate([1, 3, 5, 10, 15, 20]):
    # Entrenar modelo
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)

    # Predecir
```

```
    Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # Graficar
    axes[idx].contourf(xx, yy, Z, alpha=0.4, cmap='RdYlBu_r')
    axes[idx].scatter(X_train_scaled[:, 0], X_train_scaled[:, 1],
                      c=y_train, cmap='RdYlBu_r',
                      edgecolors='black', s=50)

    acc = knn.score(X_test_scaled, y_test)
    axes[idx].set_title(f'k={k}, Accuracy={acc:.3f}')
    axes[idx].set_xlabel('Feature 1 (escalado)')
    axes[idx].set_ylabel('Feature 2 (escalado)')
    axes[idx].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('knn_comparison.png', dpi=300, bbox_inches='tight')
print("Comparación guardada como 'knn_comparison.png'")
```

4 Árboles de decisión

4.1 Fundamento del algoritmo

Los árboles de decisión son modelos no paramétricos que segmentan recursivamente el espacio de características mediante reglas de decisión simples, creando una estructura jerárquica tipo árbol. Son altamente interpretables y capturan relaciones no lineales complejas.

4.1.1 Estructura del árbol

Un árbol de decisión consiste en:
Nodo raíz: Contiene todos los datos
Nodos internos: Representan decisiones basadas en características

- Test en una característica: $X_j \leq t$
- División binaria del espacio

Hojas (nodos terminales): Contienen predicciones

- Clasificación: Clase mayoritaria en la hoja
- Probabilidad: Distribución empírica de clases

Ramas: Conectan nodos, representan resultados de tests

4.1.2 Algoritmo de construcción recursiva

El algoritmo CART (Classification and Regression Trees) construye el árbol mediante particiones binarias recursivas:
Paso 1 - Selección de división óptima:
Para cada nodo, encontrar la mejor división (j, t) que minimiza una función de impureza:

$$(j^*, t^*) = \arg\min_{j,t} [N_L \cdot I(L) + N_R \cdot I(R)]$$

donde:

- j: Índice de característica
- t: Umbral de división
- N_L, N_R: Número de muestras en nodo izquierdo/derecho
- $I(L), I(R)$: Impureza de nodos hijo

Paso 2 - División:
Dividir datos según $X_j \leq t$:

- Izquierda: $\{(\mathbf{X}_i, Y_i) : X_{ij} \leq t\}$
- Derecha: $\{(\mathbf{X}_i, Y_i) : X_{ij} > t\}$

Paso 3 - Recursión:
Repetir para cada nodo hijo hasta cumplir criterio de parada.

4.1.3 Criterios de impureza

1. Índice de Gini (Gini Impurity):

$$\text{Gini}(N) = 1 - \sum_{k=1}^{K} p_k^2$$

donde p_k es la proporción de muestras de clase k en el nodo N.

Interpretación: Probabilidad de clasificar incorrectamente una observación aleatoria si se etiqueta aleatoriamente según distribución del nodo.

Propiedades:

- Mínimo: 0 (nodo puro, todas las muestras de una clase)
- Máximo: $1 - 1/K$ (distribución uniforme sobre K clases)
- Para $K = 2$: Máximo en $p = 0.5$, Gini = 0.5

2. Entropía (Entropy):

$$\text{Entropy}(N) = -\sum_{k=1}^{K} p_k \log_2(p_k)$$

(por convención, $0 \log 0 = 0$)

Interpretación: Cantidad promedio de información (en bits) necesaria para codificar la clase.

Propiedades:

- Mínimo: 0 (nodo puro)
- Máximo: $\log_2(K)$ (distribución uniforme)
- Para $K = 2$: Máximo en $p = 0.5$, Entropy = 1

Ganancia de información (Information Gain):

$$\text{IG} = \text{Entropy}(\text{padre}) - \left[\frac{N_L}{N}\text{Entropy}(L) + \frac{N_R}{N}\text{Entropy}(R)\right]$$

3. Error de clasificación:

$$\text{Error}(N) = 1 - \max_k p_k$$

Menos sensible que Gini y Entropy; menos usado en práctica.

4.1.4 Comparación Gini vs Entropy

Para clasificación binaria:

$$\text{Gini} = 2p(1-p)$$

$$\text{Entropy} = -p\log_2(p) - (1-p)\log_2(1-p)$$

Diferencias:

- Entropy penaliza más desigualdades
- Gini es más rápido de calcular (no requiere logaritmo)
- En práctica, producen árboles similares

4.1.5 Criterios de parada

El proceso de división se detiene cuando:

1. **Pureza perfecta**: Todas las muestras en el nodo pertenecen a la misma clase
2. **Profundidad máxima** (`max_depth`): Se alcanza profundidad especificada
3. **Muestras mínimas para dividir** (`min_samples_split`):

$$N_{\text{nodo}} < \text{min_samples_split}$$

4. **Muestras mínimas en hoja** (`min_samples_leaf`): División resultaría en hoja con menos muestras que umbral
5. **Ganancia de información insuficiente** (`min_impurity_decrease`):

$$\Delta\text{Impurity} < \text{threshold}$$

6. **Número de hojas máximo** (`max_leaf_nodes`): Se alcanza número especificado de hojas

4.1.6 Predicción

Para clasificar una nueva observación $\mathbf{X}_0$:
Paso 1: Comenzar en raíz
Paso 2: Seguir ramas según tests:

- Si $X_{0j} \leq t$: ir a nodo izquierdo
- Si $X_{0j} > t$: ir a nodo derecho

Paso 3: Al llegar a hoja, predecir:
Clase:

$$\hat{Y}_0 = \arg\max_k p_k^{(\text{hoja})}$$

Probabilidad:

$$P(Y = k|\mathbf{X}_0) = p_k^{(\text{hoja})} = \frac{N_k^{(\text{hoja})}}{N^{(\text{hoja})}}$$

4.1.7 Fronteras de decisión

Las fronteras son **hiperplanos perpendiculares a los ejes** (axis-aligned):

- Cada división crea frontera perpendicular a un eje de característica
- Partición recursiva del espacio en regiones rectangulares
- Dentro de cada región: predicción constante

Limitación: No puede representar eficientemente fronteras oblicuas (ej: $X_1 + X_2 = 0$).

4.1.8 Importancia de características

La importancia de característica j se calcula como:

$$\text{Importance}(j) = \sum_{N:\text{usa } X_j} \Delta\text{Impurity}(N) \cdot \frac{N_{\text{samples}}(N)}{N_{\text{total}}}$$

Suma ponderada de reducción de impureza sobre todos los nodos que usan X_j.
Normalización:

$$\sum_{j=1}^{p} \text{Importance}(j) = 1$$

4.1.9 Poda del árbol (Pruning)

Para controlar overfitting:
1. Pre-pruning (early stopping): Detener crecimiento usando criterios de parada estrictos

2. Post-pruning (pruning): Construir árbol completo, luego podar:
Cost-Complexity Pruning:

$$R_\alpha(T) = R(T) + \alpha|T|$$

donde:

- $R(T)$: Error de clasificación del árbol
- $|T|$: Número de hojas
- $\alpha \geq 0$: Parámetro de complejidad

Seleccionar α que minimiza error de validación cruzada.

4.1.10 Ventajas

1. **Interpretabilidad**: Fácil de entender y visualizar
2. **No requiere preprocesamiento**: No necesita estandarización ni normalización
3. **Maneja datos mixtos**: Características numéricas y categóricas
4. **No paramétrico**: No asume distribución de datos
5. **Captura no linealidades**: Automáticamente modela interacciones complejas
6. **Robustez a outliers**: En variables independientes (no en Y)
7. **Selección implícita de features**: Usa solo características informativas
8. **Manejo de missing values**: Algoritmos con estrategias para datos faltantes

4.1.11 Limitaciones

1. **Overfitting**: Tendencia a ajustar demasiado datos de entrenamiento
    - Árboles profundos memorizan ruido
2. **Inestabilidad**: Alta varianza
    - Pequeños cambios en datos → árbol completamente diferente
    - No robusto
3. **Sesgo con clases desbalanceadas**: Tiende a favorecer clase mayoritaria
4. **Fronteras axis-aligned**: No eficiente para fronteras oblicuas
5. **Greedy algorithm**: Optimización local en cada división
    - No garantiza árbol globalmente óptimo
6. **Discontinuidades**: Predicciones cambian abruptamente en fronteras

4.1.12 Hiperparámetros principales

Parámetro	Descripción	Efecto
`max_depth`	Profundidad máxima	Mayor → Más complejo, overfitting
`min_samples_split`	Muestras mínimas para dividir	Mayor → Más simple
`min_samples_leaf`	Muestras mínimas en hoja	Mayor → Más suave
`max_features`	Features a considerar	Menor → Más diversidad
`max_leaf_nodes`	Número máximo de hojas	Menor → Más simple
`min_impurity_decrease`	Ganancia mínima	Mayor → Más conservador

4.1.13 Aplicaciones en economía

1. **Credit scoring**: Reglas claras para aprobación/rechazo de créditos
2. **Segmentación de clientes**: Identificar grupos con características similares
3. **Detección de fraude**: Reglas para identificar transacciones sospechosas
4. **Predicción de churn**: Identificar clientes en riesgo de abandonar
5. **Pricing dinámico**: Reglas para ajustar precios según características
6. **Evaluación de riesgo**: Clasificación de riesgo crediticio o empresarial
7. **Decisiones de inversión**: Reglas para compra/venta de activos

4.1.14 Comparación con otros métodos

Aspecto	Decision Trees	Logistic Regression	KNN
Interpretabilidad	Alta	Media	Baja
Captura no linealidades	Sí	No (sin transformaciones)	Sí
Preprocesamiento	Mínimo	Requiere estandarización	Requiere estandarización
Overfitting	Alto	Bajo-Medio	Medio-Alto
Estabilidad	Baja	Alta	Media
Velocidad predicción	Rápida	Rápida	Lenta

4.1.15 Extensiones y variantes

1. Árboles de regresión (CART): Predicción de valores continuos
2. Random Forests: Ensemble de árboles (siguiente sección)
3. Gradient Boosted Trees: Ensemble secuencial (XGBoost, LightGBM)
4. Conditional Inference Trees: Usa tests estadísticos para divisiones
5. Árboles multivariados: Divisiones oblicuas (no axis-aligned)

4.2 Implementación básica

```
# Entrenar árbol de decisión
modelo_tree = DecisionTreeClassifier(
    max_depth=3,
    min_samples_split=10,
    min_samples_leaf=5,
    random_state=42
```

```
)

modelo_tree.fit(X_train, y_train)

# Evaluar
y_pred_tree = modelo_tree.predict(X_test)
acc_tree = accuracy_score(y_test, y_pred_tree)

print("ÁRBOL DE DECISIÓN")
print("=" * 70)
print(f"Profundidad del árbol: {modelo_tree.get_depth()}")
print(f"Número de hojas: {modelo_tree.get_n_leaves()}")
print(f"Accuracy en test: {acc_tree:.4f} ({acc_tree*100:.1f}%)")

# Importancia de características
importancias = pd.DataFrame({
    'caracteristica': ['Largo sépalo', 'Ancho sépalo'],
    'importancia': modelo_tree.feature_importances_
}).sort_values('importancia', ascending=False)

print(f"\nImportancia de características:")
for _, row in importancias.iterrows():
    print(f"  {row['caracteristica']}: {row['importancia']:.4f}")
```

4.3 Visualización del árbol

```
# Visualizar árbol
plt.figure(figsize=(20, 10))
tree.plot_tree(
    modelo_tree,
    feature_names=['Largo sépalo', 'Ancho sépalo'],
    class_names=['No Virginica', 'Virginica'],
    filled=True,
    rounded=True,
    fontsize=10
)
plt.title('Árbol de Decisión - Clasificación de Iris',
          fontsize=16, fontweight='bold', pad=20)
plt.tight_layout()
plt.savefig('decision_tree.png', dpi=300, bbox_inches='tight')
print("\nÁrbol visualizado y guardado como 'decision_tree.png'")
```

4.4 Efecto de la profundidad

```
# Evaluar diferentes profundidades
profundidades = range(1, 11)
```

```
resultados_depth = []

print("\n\nEFECTO DE LA PROFUNDIDAD DEL ÁRBOL")
print("=" * 70)
print(f"{'Profundidad':<12} {'Accuracy Train':<15} {'Accuracy Test':<15} {'Hojas':<10
print("-" * 70)

for depth in profundidades:
    tree_model = DecisionTreeClassifier(max_depth=depth, random_state=42)
    tree_model.fit(X_train, y_train)

    acc_train = tree_model.score(X_train, y_train)
    acc_test = tree_model.score(X_test, y_test)
    n_hojas = tree_model.get_n_leaves()

    resultados_depth.append({
        'depth': depth,
        'acc_train': acc_train,
        'acc_test': acc_test,
        'n_leaves': n_hojas
    })

    print(f"{depth:<12} {acc_train:<15.4f} {acc_test:<15.4f} {n_hojas:<10}")

# Graficar
df_depth = pd.DataFrame(resultados_depth)

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Accuracy
axes[0].plot(df_depth['depth'], df_depth['acc_train'],
             'o-', label='Training', linewidth=2)
axes[0].plot(df_depth['depth'], df_depth['acc_test'],
             's-', label='Test', linewidth=2)
axes[0].set_xlabel('Profundidad máxima')
axes[0].set_ylabel('Accuracy')
axes[0].set_title('Accuracy vs Profundidad del Árbol')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Complejidad
axes[1].plot(df_depth['depth'], df_depth['n_leaves'],
             'o-', color='green', linewidth=2)
axes[1].set_xlabel('Profundidad máxima')
axes[1].set_ylabel('Número de hojas')
axes[1].set_title('Complejidad del Árbol')
axes[1].grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.savefig('tree_depth_analysis.png', dpi=300, bbox_inches='tight')
print("\nAnálisis guardado como 'tree_depth_analysis.png'")
```

5 Random Forests

5.1 Fundamento del algoritmo

Random Forests (Bosques Aleatorios) es un método de ensemble que combina múltiples árboles de decisión para producir un clasificador más robusto y preciso. Fue propuesto por Leo Breiman en 2001 y se basa en el principio de "la sabiduría de las multitudes".

5.1.1 Concepto central: Ensemble Learning

Idea fundamental: Combinar predicciones de múltiples modelos débiles para crear un modelo fuerte.

Analogía: Consultar a múltiples expertos y promediar sus opiniones suele dar mejor resultado que confiar en uno solo.

5.1.2 Algoritmo Random Forest

Entrada:

- Dataset $\mathcal{D} = \{(\mathbf{X}_i, Y_i)\}_{i=1}^n$
- Número de árboles B
- Número de características por división m

Proceso:
Para $b = 1, \ldots, B$:
Paso 1 - Bootstrap: Crear muestra aleatoria con reemplazo

$$\mathcal{D}_b = \{\text{muestra } n \text{ observaciones de } \mathcal{D} \text{ con reemplazo}\}$$

Aproximadamente 63.2% de observaciones únicas en cada muestra (algunas repetidas, otras excluidas).

Paso 2 - Entrenar árbol con randomización adicional:
Para cada nodo del árbol b:

- Seleccionar aleatoriamente m características de las p disponibles
- Encontrar mejor división usando solo estas m características
- Dividir nodo
- Repetir recursivamente

Paso 3 - Crecer árbol completo: Sin poda (o con poda mínima). Cada árbol tiene alta varianza pero bajo sesgo.

Predicción:
Para nueva observación $\mathbf{X}_0$:
Clasificación (votación mayoritaria):

$$\hat{Y}_0 = \text{moda}\{\hat{Y}_0^{(1)}, \hat{Y}_0^{(2)}, \ldots, \hat{Y}_0^{(B)}\}$$

Probabilidad:

$$\hat{P}(Y = k|\mathbf{X}_0) = \frac{1}{B}\sum_{b=1}^{B} \mathbb{1}(\hat{Y}_0^{(b)} = k)$$

5.1.3 Dos fuentes de aleatoriedad

Random Forests introduce dos niveles de aleatoriedad:
1. Bootstrap (Bagging):

- Diferentes muestras de entrenamiento para cada árbol
- Reduce varianza al promediar predictores correlacionados

2. Random Feature Selection:

- Diferentes subsets de características en cada división
- **Decorrelaciona** los árboles
- Típicamente $m = \sqrt{p}$ para clasificación

Sin (2), los árboles serían muy similares (correlacionados), y el promedio no reduciría tanto la varianza.

5.1.4 Relación con Bagging

Bagging (Bootstrap Aggregating): Caso especial de Random Forest donde $m = p$
Random Forest = Bagging + Random Feature Selection
La selección aleatoria de features es la innovación clave que mejora Bagging.

5.1.5 Out-of-Bag (OOB) Error

Para cada árbol b, aproximadamente 37% de observaciones no están en $\mathcal{D}_b$.
Estas observaciones **Out-of-Bag** pueden usarse como conjunto de validación interno:

$$\text{OOB Error} = \frac{1}{n}\sum_{i=1}^{n} \mathbb{1}\left(Y_i \neq \frac{1}{|B_i|}\sum_{b \in B_i} \hat{Y}_i^{(b)}\right)$$

donde B_i es el conjunto de árboles que no usaron observación i.
Ventaja: No requiere conjunto de validación separado.

5.1.6 Importancia de características

Random Forests calcula dos medidas de importancia:
1. Mean Decrease Impurity (MDI):

$$\text{Importance}(X_j) = \frac{1}{B}\sum_{b=1}^{B} \sum_{N \in T_b : \text{usa } X_j} \Delta I(N) \cdot \frac{n_N}{n}$$

Promedio sobre todos los árboles de la reducción de impureza cuando se usa X_j.
2. Mean Decrease Accuracy (MDA) (Permutation Importance):
Para cada característica X_j:

1. Calcular OOB error original
2. Permutar aleatoriamente valores de X_j en OOB
3. Recalcular OOB error
4. Importancia = Incremento en error

$$\text{Importance}(X_j) = \text{OOB Error}_{\text{permutado}} - \text{OOB Error}_{\text{original}}$$

Interpretación: Si X_j es importante, permutarla aumentará significativamente el error.

5.1.7 Propiedades teóricas

Teorema (Breiman):
Bajo ciertas condiciones, cuando $B \to \infty$:

$$\lim_{B\to\infty} \text{Error}_{RF} = \rho \cdot \bar{\sigma}^2 \cdot (1 - s^2)$$

donde:

- ρ: Correlación promedio entre árboles
- $\bar{\sigma}^2$: Varianza promedio de cada árbol
- s: Fuerza promedio de cada árbol

Implicaciones:

1. **Aumentar B**: Siempre ayuda (no overfitting)
2. **Reducir ρ** (decorrelación): Ayuda → Random feature selection
3. **Aumentar fuerza árboles**: Ayuda → Usar más características

5.1.8 Hiperparámetros principales

Parámetro	Descripción	Valor típico	Efecto
`n_estimators`	Número de árboles	100-500	Más = mejor (con límite)
`max_features`	Features por división	$\sqrt{p}$ (clasificación)	Controla correlación
`max_depth`	Profundidad máxima	None (completo)	Controla sesgo-varianza
`min_samples_split`	Mínimo para dividir	2	Mayor = más simple
`min_samples_leaf`	Mínimo en hoja	1	Mayor = más suave
`bootstrap`	Usar bootstrap	True	False = usar todo dataset

5.1.9 Ventajas

1. **Reduce overfitting**: Vs árbol único, por promediado
2. **Alta precisión**: Entre los mejores algoritmos out-of-the-box
3. **Robusto**: Menos sensible a hiperparámetros que otros métodos
4. **Maneja alta dimensionalidad**: Funciona bien incluso con p grande
5. **Importancia de features**: Proporciona ranking de características
6. **No requiere estandarización**: Como árboles individuales
7. **Paralelizable**: Árboles se entrenan independientemente

8. **OOB error**: Validación interna sin conjunto separado
9. **Maneja missing values**: Estrategias incorporadas
10. **Versátil**: Funciona bien en diversos problemas

5.1.10 Limitaciones

1. **Menos interpretable**: Difícil entender modelo (caja negra)
2. **Costo computacional**:
   - Entrenamiento: $O(B \cdot n \log n \cdot m)$
   - Predicción: $O(B \cdot \log n)$ Más lento que un árbol único
3. **Memoria**: Requiere almacenar B árboles
4. **Extrapolación limitada**: No predice fuera del rango de datos de entrenamiento
5. **Sesgo con clases desbalanceadas**: Puede favorecer clase mayoritaria
6. **No captura tendencias lineales simples**: A veces un modelo simple es mejor

5.1.11 Comparación con otros métodos

Aspecto	Random Forest	Decision Tree	Gradient Boosting
Interpretabilidad	Baja	Alta	Baja
Accuracy	Alta	Media	Muy alta
Overfitting	Bajo	Alto	Medio (con tuning)
Velocidad entrenamiento	Media	Rápida	Lenta
Velocidad predicción	Media	Rápida	Media
Hiperparámetros	Pocos críticos	Pocos	Muchos críticos
Robustez	Alta	Baja	Media

5.1.12 Selección de hiperparámetros

Estrategia típica:

1. **n_estimators**: Comenzar con 100, aumentar hasta que performance se estabilice
2. **max_features**:
   - Clasificación: $\sqrt{p}$
   - Regresión: $p/3$
   - Probar también $\log_2(p)$ y $p/2$
3. **max_depth**: Comenzar sin límite, reducir si overfitting
4. **min_samples_split** y **min_samples_leaf**: Usar valores por defecto inicialmente

Grid Search sobre:

- `n_estimators`: [50, 100, 200, 500]
- `max_features`: [‘sqrt’, ‘log2’, 0.3]
- `max_depth`: [None, 10, 20, 30]
- `min_samples_split`: [2, 5, 10]

5.1.13 Aplicaciones en economía

1. **Credit scoring**: Predicción de incumplimiento con alta precisión
2. **Predicción de precios**:
    - Inmuebles
    - Acciones
    - Commodities
3. **Detección de fraude**: Identificar transacciones fraudulentas
4. **Churn prediction**: Predecir abandono de clientes
5. **Segmentación avanzada**: Clasificar clientes en múltiples categorías
6. **Predicción de quiebra empresarial**: Clasificación de riesgo
7. **Trading algorítmico**: Señales de compra/venta
8. **Risk management**: Evaluación de riesgo de portafolios

5.1.14 Variantes y extensiones

1. Extremely Randomized Trees (Extra Trees):

- Divisiones completamente aleatorias (no optimizadas)
- Más rápido, más diversidad

2. Balanced Random Forest:

- Balanceo de clases en cada árbol
- Mejor para datasets desbalanceados

3. Isolation Forest:

- Para detección de anomalías
- Aísla outliers

4. Quantile Regression Forest:

- Predice distribución completa, no solo media
- Útil para intervalos de confianza

5.2 Implementación

```
# Entrenar Random Forest
modelo_rf = RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    min_samples_split=10,
    min_samples_leaf=5,
    random_state=42,
    n_jobs=-1  # Usar todos los procesadores
)

modelo_rf.fit(X_train, y_train)

# Evaluar
y_pred_rf = modelo_rf.predict(X_test)
acc_rf = accuracy_score(y_test, y_pred_rf)

print("RANDOM FOREST")
print("=" * 70)
print(f"Número de árboles: {modelo_rf.n_estimators}")
print(f"Accuracy en test: {acc_rf:.4f} ({acc_rf*100:.1f}%)")

# Importancia de características
importancias_rf = pd.DataFrame({
    'caracteristica': ['Largo sépalo', 'Ancho sépalo'],
    'importancia': modelo_rf.feature_importances_,
    'std': np.std([tree.feature_importances_
                   for tree in modelo_rf.estimators_], axis=0)
}).sort_values('importancia', ascending=False)

print(f"\nImportancia de características:")
for _, row in importancias_rf.iterrows():
    print(f"  {row['caracteristica']}: {row['importancia']:.4f} (±{row['std']:.4f})")
```

5.3 Efecto del número de árboles

```
# Evaluar diferentes números de árboles
n_trees_list = [1, 5, 10, 20, 50, 100, 200]
resultados_trees = []

print("\n\nEFECTO DEL NÚMERO DE ÁRBOLES")
print("=" * 70)
print(f"{'N Árboles':<12} {'Accuracy Train':<15} {'Accuracy Test':<15}")
print("-" * 70)

for n_trees in n_trees_list:
    rf = RandomForestClassifier(
        n_estimators=n_trees,
```

```
        max_depth=5,
        random_state=42
    )
    rf.fit(X_train, y_train)

    acc_train = rf.score(X_train, y_train)
    acc_test = rf.score(X_test, y_test)

    resultados_trees.append({
        'n_trees': n_trees,
        'acc_train': acc_train,
        'acc_test': acc_test
    })

    print(f"{n_trees:<12} {acc_train:<15.4f} {acc_test:<15.4f}")

# Graficar
df_trees = pd.DataFrame(resultados_trees)

plt.figure(figsize=(10, 6))
plt.plot(df_trees['n_trees'], df_trees['acc_train'],
         'o-', label='Training', linewidth=2, markersize=8)
plt.plot(df_trees['n_trees'], df_trees['acc_test'],
         's-', label='Test', linewidth=2, markersize=8)
plt.xlabel('Número de árboles')
plt.ylabel('Accuracy')
plt.title('Accuracy vs Número de Árboles en Random Forest')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xscale('log')
plt.tight_layout()
plt.savefig('random_forest_ntrees.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'random_forest_ntrees.png'")
```

6 Clasificación multi-clase

6.1 Dataset con 3 clases

```
# Usar todas las especies de Iris (3 clases)
X_multiclass = iris.data[:, :2]
y_multiclass = iris.target

print("CLASIFICACIÓN MULTI-CLASE")
print("=" * 70)
print(f"Número de clases: {len(np.unique(y_multiclass))}")
print(f"Clases: {iris.target_names}")
print(f"\nDistribución:")
```

```
for i, nombre in enumerate(iris.target_names):
    count = (y_multiclass == i).sum()
    print(f"  {nombre}: {count} muestras")

# Dividir datos
X_train_mc, X_test_mc, y_train_mc, y_test_mc = train_test_split(
    X_multiclass, y_multiclass, test_size=0.3, random_state=42
)
```

6.2 Entrenar modelos multi-clase

```
# Entrenar cada modelo
modelos_mc = {
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=200),
    'KNN': KNeighborsClassifier(n_neighbors=5),
    'Decision Tree': DecisionTreeClassifier(max_depth=5, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42)
}

resultados_mc = {}

print("\n\nRESULTADOS CLASIFICACIÓN MULTI-CLASE")
print("=" * 70)

for nombre, modelo in modelos_mc.items():
    # Entrenar
    modelo.fit(X_train_mc, y_train_mc)

    # Predecir
    y_pred = modelo.predict(X_test_mc)

    # Evaluar
    acc = accuracy_score(y_test_mc, y_pred)

    resultados_mc[nombre] = {
        'modelo': modelo,
        'accuracy': acc
    }

    print(f"\n{nombre}:")
    print(f"  Accuracy: {acc:.4f} ({acc*100:.1f}%)")

    # Matriz de confusión
    cm = confusion_matrix(y_test_mc, y_pred)
    print(f"  Matriz de confusión:")
    print(f"  {cm}")
```

```
# Encontrar mejor modelo
mejor_modelo_mc = max(resultados_mc, key=lambda x: resultados_mc[x]['accuracy'])
print(f"\n\nMejor modelo: {mejor_modelo_mc}")
print(f"Accuracy: {resultados_mc[mejor_modelo_mc]['accuracy']:.4f}")
```

6.3 Visualización multi-clase

```
# Visualizar fronteras de decisión para cada modelo
fig, axes = plt.subplots(2, 2, figsize=(16, 14))
axes = axes.ravel()

h = 0.02
x_min, x_max = X_multiclass[:, 0].min() - 0.5, X_multiclass[:, 0].max() + 0.5
y_min, y_max = X_multiclass[:, 1].min() - 0.5, X_multiclass[:, 1].max() + 0.5
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))

for idx, (nombre, resultado) in enumerate(resultados_mc.items()):
    modelo = resultado['modelo']
    acc = resultado['accuracy']

    # Predecir para malla
    Z = modelo.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # Graficar
    axes[idx].contourf(xx, yy, Z, alpha=0.4, cmap='viridis')
    scatter = axes[idx].scatter(X_multiclass[:, 0], X_multiclass[:, 1],
                                c=y_multiclass, cmap='viridis',
                                edgecolors='black', s=50)

    axes[idx].set_xlabel('Largo del sépalo (cm)')
    axes[idx].set_ylabel('Ancho del sépalo (cm)')
    axes[idx].set_title(f'{nombre}\nAccuracy={acc:.3f}')
    axes[idx].grid(True, alpha=0.3)

# Leyenda común
plt.colorbar(scatter, ax=axes, label='Especie', ticks=[0, 1, 2])

plt.tight_layout()
plt.savefig('multiclass_comparison.png', dpi=300, bbox_inches='tight')
print("\nComparación guardada como 'multiclass_comparison.png'")
```

7 Comparación de modelos

7.1 Curvas ROC para cada modelo

```
# Calcular curvas ROC para clasificación binaria
def calcular_roc_multimodelo(modelos_dict, X_test, y_test):
    """Calcula curvas ROC para múltiples modelos"""

    plt.figure(figsize=(10, 8))

    for nombre, modelo in modelos_dict.items():
        # Obtener probabilidades
        if hasattr(modelo, 'predict_proba'):
            probs = modelo.predict_proba(X_test)[:, 1]
        else:
            probs = modelo.decision_function(X_test)

        # Calcular curva ROC
        fpr, tpr, _ = roc_curve(y_test, probs)
        roc_auc = auc(fpr, tpr)

        # Graficar
        plt.plot(fpr, tpr, linewidth=2,
                label=f'{nombre} (AUC={roc_auc:.3f})')

    # Línea diagonal
    plt.plot([0, 1], [0, 1], 'k--', linewidth=2, label='Azar (AUC=0.5)')

    plt.xlabel('False Positive Rate', fontsize=12)
    plt.ylabel('True Positive Rate', fontsize=12)
    plt.title('Curvas ROC - Comparación de Modelos',
              fontsize=14, fontweight='bold')
    plt.legend(loc='lower right')
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig('roc_comparison.png', dpi=300, bbox_inches='tight')
    print("Curvas ROC guardadas como 'roc_comparison.png'")

# Preparar modelos para comparación (clasificación binaria)
modelos_binarios = {
    'Logistic Regression': LogisticRegression(random_state=42),
    'KNN (k=5)': KNeighborsClassifier(n_neighbors=5),
    'Decision Tree': DecisionTreeClassifier(max_depth=5, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42)
}

# Entrenar modelos
for modelo in modelos_binarios.values():
```

```
    modelo.fit(X_train, y_train)

# Calcular y graficar ROC
calcular_roc_multimodelo(modelos_binarios, X_test, y_test)
```

7.2 Tabla comparativa de métricas

```
# Compilar todas las métricas
print("\n\nTABLA COMPARATIVA DE MODELOS")
print("=" * 70)

resultados_comparacion = []

for nombre, modelo in modelos_binarios.items():
    # Predicciones
    y_pred = modelo.predict(X_test)

    # Probabilidades
    if hasattr(modelo, 'predict_proba'):
        probs = modelo.predict_proba(X_test)[:, 1]
    else:
        probs = modelo.decision_function(X_test)

    # Métricas
    from sklearn.metrics import precision_score, recall_score, f1_score

    acc = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    # ROC AUC
    fpr, tpr, _ = roc_curve(y_test, probs)
    roc_auc = auc(fpr, tpr)

    resultados_comparacion.append({
        'Modelo': nombre,
        'Accuracy': acc,
        'Precision': precision,
        'Recall': recall,
        'F1-Score': f1,
        'AUC': roc_auc
    })

# Crear DataFrame
df_comparacion = pd.DataFrame(resultados_comparacion)
df_comparacion = df_comparacion.round(4)
```

```
print(df_comparacion.to_string(index=False))

# Guardar
df_comparacion.to_csv('comparacion_modelos.csv', index=False)
print("\n\nTabla guardada como 'comparacion_modelos.csv'")
```

8 Aplicaciones económicas

8.1 Aplicación 1: Clasificación de riesgo crediticio

```
print("\n\n\nAPLICACIÓN: CLASIFICACIÓN DE RIESGO CREDITICIO")
print("=" * 70)

# Generar datos sintéticos
np.random.seed(42)
n_clientes = 2000

datos_credito = pd.DataFrame({
    'edad': np.random.randint(18, 70, n_clientes),
    'ingreso_anual': np.random.uniform(10000, 150000, n_clientes),
    'monto_solicitado': np.random.uniform(5000, 200000, n_clientes),
    'antiguedad_laboral': np.random.randint(0, 30, n_clientes),
    'num_creditos_previos': np.random.randint(0, 10, n_clientes),
    'ratio_deuda_ingreso': np.random.uniform(0, 1.5, n_clientes),
    'tiene_hipoteca': np.random.choice([0, 1], n_clientes),
    'score_crediticio': np.random.randint(300, 850, n_clientes)
})

# Variable objetivo: riesgo (0=bajo, 1=medio, 2=alto)
score_riesgo = (
    -datos_credito['score_crediticio'] * 0.003 +
    datos_credito['ratio_deuda_ingreso'] * 2 +
    -datos_credito['antiguedad_laboral'] * 0.05 +
    datos_credito['num_creditos_previos'] * 0.2 +
    np.random.normal(0, 0.5, n_clientes)
)

# Clasificar en 3 niveles
datos_credito['riesgo'] = pd.cut(
    score_riesgo,
    bins=3,
    labels=['Bajo', 'Medio', 'Alto']
)

# Convertir a numérico
datos_credito['riesgo_num'] = datos_credito['riesgo'].map({
    'Bajo': 0,
```

```
    'Medio': 1,
    'Alto': 2
})

print(f"\nDataset de crédito:")
print(f"  Total clientes: {n_clientes:,}")
print(f"\nDistribución de riesgo:")
print(datos_credito['riesgo'].value_counts())

# Preparar datos
X_credito = datos_credito.drop(['riesgo', 'riesgo_num'], axis=1)
y_credito = datos_credito['riesgo_num']

X_train_cr, X_test_cr, y_train_cr, y_test_cr = train_test_split(
    X_credito, y_credito, test_size=0.3, random_state=42, stratify=y_credito
)

# Estandarizar
scaler_credito = StandardScaler()
X_train_cr_scaled = scaler_credito.fit_transform(X_train_cr)
X_test_cr_scaled = scaler_credito.transform(X_test_cr)

# Entrenar modelos
print("\n\nENTRENAMIENTO DE MODELOS")
print("=" * 70)

modelos_credito = {
    'Random Forest': RandomForestClassifier(n_estimators=200, max_depth=10, random_st
    'Decision Tree': DecisionTreeClassifier(max_depth=8, random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=10),
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42)
}

for nombre, modelo in modelos_credito.items():
    # Entrenar
    if nombre in ['KNN', 'Logistic Regression']:
        modelo.fit(X_train_cr_scaled, y_train_cr)
        y_pred = modelo.predict(X_test_cr_scaled)
    else:
        modelo.fit(X_train_cr, y_train_cr)
        y_pred = modelo.predict(X_test_cr)

    # Evaluar
    acc = accuracy_score(y_test_cr, y_pred)

    print(f"\n{nombre}:")
    print(f"  Accuracy: {acc:.4f} ({acc*100:.1f}%)")
```

```
    # Reporte por clase
    print(f"\n  Reporte de clasificación:")
    print(classification_report(
        y_test_cr, y_pred,
        target_names=['Bajo', 'Medio', 'Alto'],
        digits=3
    ))

# Importancia de características (Random Forest)
print("\n\nIMPORTANCIA DE CARACTERÍSTICAS (Random Forest)")
print("=" * 70)

rf_final = modelos_credito['Random Forest']
importancias = pd.DataFrame({
    'caracteristica': X_credito.columns,
    'importancia': rf_final.feature_importances_
}).sort_values('importancia', ascending=False)

for _, row in importancias.iterrows():
    print(f"  {row['caracteristica']:<25} {row['importancia']:.4f} {' ' * int(row['imp
```

8.2 Aplicación 2: Segmentación de clientes

```
print("\n\n\nAPLICACIÓN: SEGMENTACIÓN DE CLIENTES")
print("=" * 70)

# Generar datos de clientes
np.random.seed(42)
n_clientes_seg = 1500

datos_clientes = pd.DataFrame({
    'edad': np.random.randint(18, 80, n_clientes_seg),
    'ingreso_mensual': np.random.uniform(1000, 20000, n_clientes_seg),
    'gasto_mensual': np.random.uniform(500, 15000, n_clientes_seg),
    'num_compras_mes': np.random.randint(0, 30, n_clientes_seg),
    'antiguedad_meses': np.random.randint(1, 120, n_clientes_seg),
    'ticket_promedio': np.random.uniform(20, 500, n_clientes_seg),
    'visitas_web': np.random.randint(0, 100, n_clientes_seg)
})

# Calcular características adicionales
datos_clientes['frecuencia_compra'] = datos_clientes['num_compras_mes'] / 4
datos_clientes['ratio_gasto_ingreso'] = datos_clientes['gasto_mensual'] / datos_clien

# Crear segmentos (0=Bronce, 1=Plata, 2=Oro, 3=Platino)
score_valor = (
```

```
    datos_clientes['gasto_mensual'] * 0.0003 +
    datos_clientes['num_compras_mes'] * 0.1 +
    datos_clientes['ticket_promedio'] * 0.005 +
    np.random.normal(0, 0.3, n_clientes_seg)
)

datos_clientes['segmento'] = pd.cut(
    score_valor,
    bins=4,
    labels=['Bronce', 'Plata', 'Oro', 'Platino']
)

datos_clientes['segmento_num'] = datos_clientes['segmento'].map({
    'Bronce': 0,
    'Plata': 1,
    'Oro': 2,
    'Platino': 3
})

print(f"\nDistribución de segmentos:")
print(datos_clientes['segmento'].value_counts().sort_index())

# Preparar datos
features_seg = ['edad', 'ingreso_mensual', 'gasto_mensual', 'num_compras_mes',
                'antiguedad_meses', 'ticket_promedio', 'visitas_web',
                'frecuencia_compra', 'ratio_gasto_ingreso']

X_seg = datos_clientes[features_seg]
y_seg = datos_clientes['segmento_num']

X_train_seg, X_test_seg, y_train_seg, y_test_seg = train_test_split(
    X_seg, y_seg, test_size=0.3, random_state=42, stratify=y_seg
)

# Entrenar Random Forest (mejor modelo típicamente)
print("\n\nMODELO DE SEGMENTACIÓN")
print("=" * 70)

rf_segmentacion = RandomForestClassifier(
    n_estimators=200,
    max_depth=12,
    min_samples_split=20,
    random_state=42
)

rf_segmentacion.fit(X_train_seg, y_train_seg)
```

```
# Evaluar
y_pred_seg = rf_segmentacion.predict(X_test_seg)
acc_seg = accuracy_score(y_test_seg, y_pred_seg)

print(f"Accuracy: {acc_seg:.4f} ({acc_seg*100:.1f}%)")

# Matriz de confusión
cm_seg = confusion_matrix(y_test_seg, y_pred_seg)
print(f"\nMatriz de confusión:")
print(f"                    Predicho")
print(f"             Bronce  Plata   Oro  Platino")
print(f"Real Bronce    {cm_seg[0, 0]:4d}   {cm_seg[0, 1]:4d}  {cm_seg[0, 2]:4d}    {c
print(f"     Plata     {cm_seg[1, 0]:4d}   {cm_seg[1, 1]:4d}  {cm_seg[1, 2]:4d}    {c
print(f"     Oro       {cm_seg[2, 0]:4d}   {cm_seg[2, 1]:4d}  {cm_seg[2, 2]:4d}    {c
print(f"     Platino   {cm_seg[3, 0]:4d}   {cm_seg[3, 1]:4d}  {cm_seg[3, 2]:4d}    {c

# Características más importantes
print(f"\n\nCARACTERÍSTICAS MÁS IMPORTANTES PARA SEGMENTACIÓN")
print("=" * 70)

importancias_seg = pd.DataFrame({
    'caracteristica': features_seg,
    'importancia': rf_segmentacion.feature_importances_
}).sort_values('importancia', ascending=False)

for _, row in importancias_seg.iterrows():
    barra = ' ' * int(row['importancia'] * 100)
    print(f"  {row['caracteristica']:<25} {row['importancia']:.4f} {barra}")

print(f"\n\nAPLICACIÓN PRÁCTICA:")
print("Este modelo permite:")
print("  1. Clasificar automáticamente nuevos clientes")
print("  2. Personalizar ofertas según segmento")
print("  3. Identificar clientes con potencial de upgrade")
print("  4. Optimizar estrategias de retención")
```

9 Ejercicios prácticos

9.1 Ejercicio: Optimización de hiperparámetros

```
print("\n\n\nEJERCICIO: OPTIMIZACIÓN DE HIPERPARÁMETROS")
print("=" * 70)

# Grid search manual para Random Forest
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 15, None],
```

```
    'min_samples_split': [2, 10, 20]
}

mejores_params = None
mejor_score = 0

print("\nBuscando mejores hiperparámetros...")
print(f"Total combinaciones: {len(param_grid['n_estimators']) * len(param_grid['max_d

resultados_grid = []

for n_est in param_grid['n_estimators']:
    for max_d in param_grid['max_depth']:
        for min_split in param_grid['min_samples_split']:

            # Entrenar modelo
            rf = RandomForestClassifier(
                n_estimators=n_est,
                max_depth=max_d,
                min_samples_split=min_split,
                random_state=42
            )

            # Validación cruzada
            scores = cross_val_score(rf, X_train, y_train, cv=5)
            score_mean = scores.mean()

            resultados_grid.append({
                'n_estimators': n_est,
                'max_depth': max_d if max_d else 'None',
                'min_samples_split': min_split,
                'cv_score': score_mean
            })

            if score_mean > mejor_score:
                mejor_score = score_mean
                mejores_params = {
                    'n_estimators': n_est,
                    'max_depth': max_d,
                    'min_samples_split': min_split
                }

print(f"\n\nMEJORES HIPERPARÁMETROS")
print("=" * 70)
print(f"  n_estimators: {mejores_params['n_estimators']}")
print(f"  max_depth: {mejores_params['max_depth']}")
print(f"  min_samples_split: {mejores_params['min_samples_split']}")
```

```
print(f"  CV Score: {mejor_score:.4f}")

# Entrenar modelo final
rf_optimo = RandomForestClassifier(**mejores_params, random_state=42)
rf_optimo.fit(X_train, y_train)

acc_optimo = rf_optimo.score(X_test, y_test)
print(f"\nAccuracy en test: {acc_optimo:.4f} ({acc_optimo*100:.1f}%)")

# Top 10 combinaciones
df_grid = pd.DataFrame(resultados_grid).sort_values('cv_score', ascending=False)
print(f"\n\nTop 10 combinaciones:")
print(df_grid.head(10).to_string(index=False))
```

10 Conclusión

En esta guía hemos explorado los principales algoritmos de clasificación:

Regresión Logística

Modelo lineal simple y rápido. Usa función sigmoide para probabilidades. Interpretable. Bueno como baseline.

K-Nearest Neighbors (KNN)

No paramétrico. Clasifica según vecinos más cercanos. Sensible a escala. Bueno para fronteras complejas.

Árboles de Decisión

Reglas interpretables. No requiere normalización. Propenso a overfitting. Base para métodos ensemble.

Random Forests

Ensemble de árboles. Reduce overfitting. Alta precisión. Menos interpretable que árbol único.

Comparación de modelos

Importancia de evaluar múltiples algoritmos. Usar validación cruzada. Considerar trade-offs interpretabilidad vs precisión.

11 Próximos pasos

En la siguiente guía (Guía 10: Métodos de ML para Regresión) exploraremos:

- Regresión lineal
- Ridge y Lasso
- Regresión polinomial
- Métricas: R^2, MSE, MAE
- Aplicaciones económicas

12 Recursos adicionales

Para profundizar en clasificación:

- Scikit-learn Documentation: scikit-learn.org
- Introduction to Statistical Learning (ISLR)
- Hands-On Machine Learning (Aurélien Géron)
- Papers en credit scoring y risk management

13 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1. Instalacion De Anaconda
2. Configurar Entorno Virtual Python Anaconda
3. 01 Introducion A La Programacion Con Python
4. 02 Variables Expresiones Y Statements Con Python
5. 03 Objetos De Python
6. 04 Ejecucion Condicional Con Python
7. 05 Iteraciones Con Python
8. 06 Funciones Con Python
9. 07 Dataframes Con Python
10. 08 Prediccion Y Metrica De Performance Con Python
11. 09 Metodos De Machine Learning Para Clasificacion Con Python
12. 10 Metodos De Machine Learning Para Regresion Con Python
13. 11 Validacion Cruzada Y Composicion Del Modelo Con Python
14. Visualizacion De Datos Con Python

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!